Watersports Operations Platform

V2 — Digital ticketing and redemption
Implementation guide with Claude Code prompts

Continues the V1 build plan · Work packages WP-22 to WP-36
Target stack: Next.js (App Router) + Supabase + Vercel
Location: Mauritius · Currency: Rs (MUR) · Timezone: Indian/Mauritius (UTC+4)
Document version 1.0 · 29 September 2026

Contents

1	How to use this document
2	The gate, and what V1 left ready
3	Locked decisions for V2
4	Architecture: the island edge
5	Schema additions
6	The redemption model
7	Working offline
8	Ticket lifecycle and validity verdicts
9	Delivery: WhatsApp and email
10	Roles and RLS changes
11	V2 scope: in, out, and definition of done
12	V2 build sequence — Claude Code prompts (WP-22 to WP-36)
13	Hardware and field conditions
14	Operations runbook additions
15	Open questions and risks
16	Appendix A — WhatsApp procurement checklist
17	Appendix B — Enumerations added in V2
18	Appendix C — The island field card
19	Appendix D — Acceptance test checklist for V2

1 How to use this document

This continues the V1 build plan without repeating it. Sections 1, 4, 9, 12 and 13 of that document — architecture, engineering standards, deployment, backups — still apply unchanged. Read this one alongside it, not instead of it.

Work packages run **WP-22 to WP-36**, picking up where V1's WP-21 ended. The same rule holds: one package per Claude Code session, paste the prompt, review the diff, run the acceptance check, commit, move on.

> **A numbering note.** If you are also building the public marketing website, its addendum package (the enquiries inbox) was numbered WP-22 in that document. It is independent of V2 and can land any time after V1. Renumber it **WP-37** so the two sequences do not collide.

> **This document assumes V1.0 of the build plan** — twenty-one work packages, with `resources` (boats) still deferred to V4. If you adopted the V1.1 revision that added the owner's day sheet (WP-17A) and moved `resources` into V1, two things here change: the redemption report in WP-34 can group by vessel as well as by location, and the island app can show which boat a party arrived on. Nothing else differs, and neither is required for V2 to work.

2 The gate, and what V1 left ready

V1's exit gate was specific, and it is worth re-reading before starting: *reception has created every real booking for ten consecutive operating days in the system, the day's cash total matched the drawer, and reception prefers it to paper.*

> **Do not start V2 because V1 compiled.** V2 puts a device in the hands of staff standing on a beach, and every redemption it records is only as trustworthy as the booking underneath it. If reception is still keeping a parallel paper book, the island app will be scanning tickets for bookings that do not represent the day's real trade, and you will have built an expensive way to reconcile fiction.

2.1 What V1 deliberately built for this phase

V2 is mostly additive because V1 left the hooks in place. Nothing below needs retrofitting.

V1 left	V2 uses it for
`booking_activities` — the expanded entitlement rows, written at booking creation (WP-06)	The thing that is actually redeemed. Because V1 wrote it from day one, every historical booking can be scanned, not just those created after V2 ships.
`tickets` with a 16-byte random `token` (WP-05)	The QR payload. Unguessable from a booking reference, which matters now that customers hold the link.
`/tickets/[token]` route, resolved by token not id, with a reserved square block (WP-18)	The QR drops into the reserved block. No layout change, no reprint of the design.
`/tickets/[token]/pdf` route handler (WP-18)	The email attachment. One source of truth for the layout rather than two.
Disabled "Send WhatsApp" and "Send email" buttons on the confirmation screen (WP-16)	They become live. The layout does not move when V2 lands, which was the point of shipping them disabled.
`activity_staff` role with no table grants (WP-02, WP-07)	The boundary already exists and is already tested. V2 grants it two functions and nothing else.
`audit_logs` and the generic trigger (WP-03)	Redemptions are audited the same way as payments, with no new plumbing.

3 Locked decisions for V2

QR payload	The ticket URL containing the opaque token. Never a name, a party size, a price or a booking reference in the code itself.
Redemption unit	**Quantity, not a boolean.** Four lunches, three now, one later. A "used" flag cannot express that, and the business case for V2 is precisely the partial case.
Over-redemption	Prevented inside a function that locks the entitlement row, not by a table constraint. See 6.2 for why no constraint can express it.
Corrections	A redemption is never deleted. A mistake is reversed by a negative-quantity row referencing the original, with a reason — the same discipline V1 applied to payments.
Offline	**The island app works offline.** Decided, not deferred. The island is precisely where connectivity fails, and redemption is the whole point of the phase.
Offline conflicts	During an outage the no-over-redemption invariant cannot be maintained. The system records what actually happened and surfaces the discrepancy, rather than silently dropping the second event.
Device identity	The device is the offline unit of accountability, bound to a staff member at the start of a shift. A redemption records both.
WhatsApp	WhatsApp Business Platform with approved templates, through a BSP or the Cloud API. Never an automated personal account.
Delivery logging	Every send attempt is logged with its provider result and delivery status. A "sent" state that is a silent failure is worse than no feature at all.
Check-in vs redemption	**Two different events, modelled separately.** A party checks in once on arrival; they redeem entitlements individually through the day. See 3.2.
Not-included rows	Every entitlement display lists activities the booking does *not* include, marked NOT INCLUDED, rather than omitting them. The original requirements asked for this and it settles arguments before they start.
Roles	A fifth role, `activity_supervisor`, is added. See 10.1 — this reverses V1's "four roles, fixed" decision, with cause.

3.2 Check-in and redemption are not the same event

The original requirements' daily operations screen asks for two different numbers on the same trip:

```
CATAMARAN — 08:00
  32 PEOPLE
  ✓ 28 checked in
  ○  4 pending
  Lunch: 32   Redeemed: 26
```

Checked in and *redeemed* are distinct facts. A party of four arrives and boards — that is one check-in event covering four people. Over the next five hours they consume a lunch each, two parasailing slots and nothing else — that is three redemption events against three entitlements.

Collapsing them loses both numbers. If check-in is modelled as "redeemed the catamaran", you cannot tell who has arrived but not yet eaten, which is exactly what the kitchen needs at 11:30. So `bookings` gains four check-in columns and the scan screen gains a CHECK IN action distinct from its REDEEM actions.

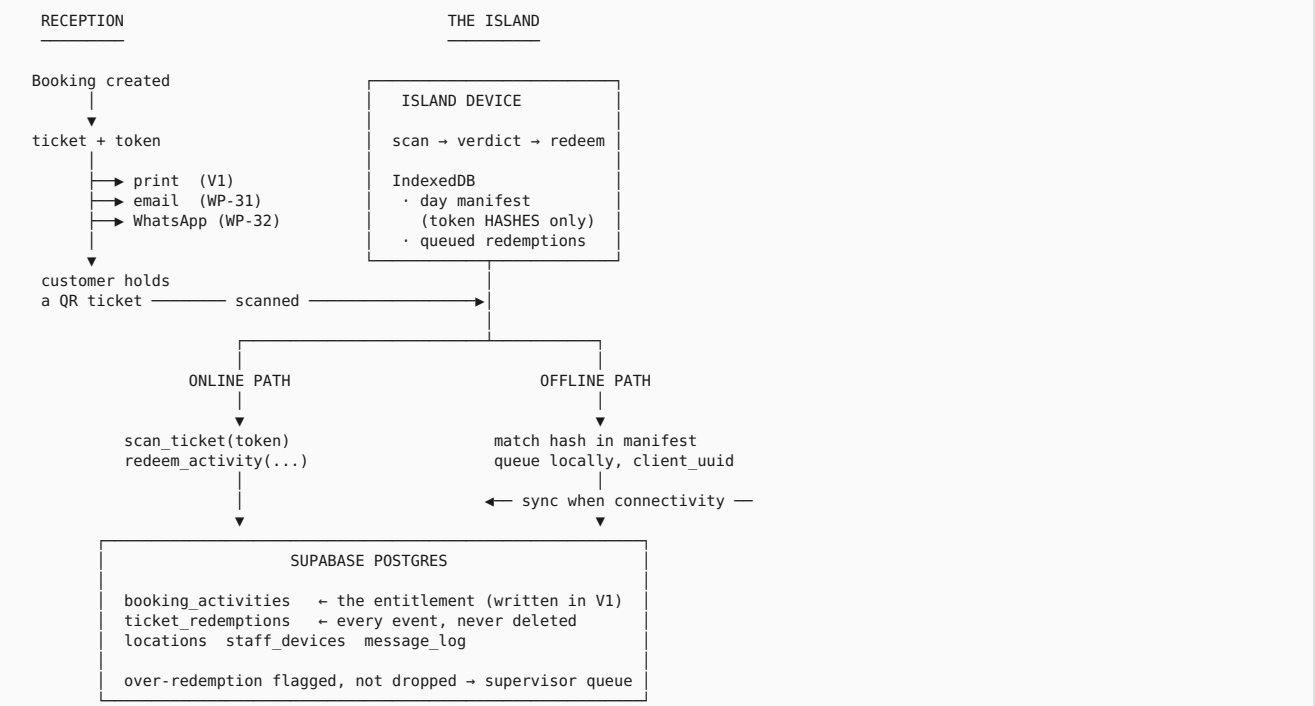
V4's departures model will later scope check-in to a specific departure rather than to the booking. The columns added here migrate into that cleanly; nothing is wasted.

3.1 Why the device is the unit of accountability offline

Online, a redemption is attributed to the signed-in user, and the server verifies that. Offline it cannot: the device holds a session it has no way to re-validate, and a session that expired an hour ago looks identical to a live one.

So V2 inverts it. A device is registered once, given a label a human would use — *Island tablet 1* — and at the start of a shift a staff member signs in on it while still on reception's wifi. Both identities are written onto every redemption: `redeemed_by` (the staff member claimed by the device) and `device_id` (verified, because the server issued the registration). When a dispute arises three weeks later, the device is the fact and the staff member is the claim. That distinction is worth having in writing before anyone needs it.

4 Architecture: the island edge



The device is a thin client with a cache, not a second system. It holds no prices, no totals, no contact details and no usable tokens. If it is lost, dropped in the lagoon or stolen, the loss is the hardware.

5 Schema additions

Table	Purpose and key columns
`locations`	Where a redemption happened. `code`, `name`, `location_type` ('reception' | 'island' | 'jetty' | 'vessel'), `is_active`. Small and stable; three or four rows.
`staff_devices`	`label` (a name a human uses), `location_id`, `device_secret_hash`, `registered_by`, `registered_at`, `last_seen_at`, `is_active`. Deactivating a device is how a lost tablet is dealt with.
`ticket_redemptions`	The event log. `booking_id`, `booking_activity_id`, `activity_id`, `quantity_redeemed` (may be negative for a reversal), `redeemed_at`, `redeemed_by`, `device_id`, `location_id`, `client_uuid` (unique — the idempotency key), `occurred_at_device`, `synced_at`, `is_reversal`, `reverses_redemption_id`, `note`, `is_over_redemption`, `reviewed_by`, `reviewed_at`.
`message_log`	Every delivery attempt. `booking_id`, `ticket_id`, `channel` ('whatsapp' | 'email'), `template`, `to_address`, `status`, `provider`, `provider_message_id`, `error_code`, `error_detail`, `attempt`, `sent_by`, `sent_at`, `delivered_at`, `read_at`, `cost_cents`.
`tickets` *(extended)*	Adds `revoked_at`, `revoked_by`, `revoke_reason`, `reissued_from_ticket_id`, `token_hash` (generated, for the offline manifest), `delivered_channels` jsonb.
`bookings` *(extended)*	Adds `checked_in_at`, `checked_in_by`, `checked_in_device_id`, `checked_in_location_id`. Nullable — a booking that never arrives stays null, which is what makes the no-show count meaningful.
`app_role` *(extended)*	Adds `activity_supervisor`.

5.1 Deliberate omissions, again

- **No `ticket_status` column.** A ticket's state is derived from its redemptions, its `revoked_at`, and its booking's status. A stored status is a cache that will disagree with the truth within a month.
- **No per-device redemption counter.** Everything is derivable from the event log. Counters drift; event logs do not.

- **No `redeemed` boolean on `booking_activities`.** Same reason. `SUM(quantity_redeemed)` against `quantity` is the only source of truth.

6 The redemption model

This is V2's equivalent of V1's pricing engine: the part most likely to be built wrong, so it is specified before any code exists.

6.1 The contract

```
redeem_activity(
  p_booking_activity_id uuid,
  p_quantity            int,      -- positive; negative only via reverse_redemption
  p_location_id         uuid,
  p_device_id           uuid,
  p_client_uuid         uuid,     -- generated on the device, before the attempt
  p_occurred_at_device  timestamptz,
  p_note                text
) returns jsonb
  → { redemption_id, entitlement_quantity, redeemed_total, remaining,
      is_over_redemption, already_applied }
```

6.2 Why no constraint can express this

The rule is *the sum of redemptions for one entitlement may not exceed its quantity*. That is a constraint across rows, and Postgres check constraints see one row at a time. An exclusion constraint cannot sum. A trigger that sums is racy without a lock. So the rule lives in one function, and the function is the only thing granted:

1. `SELECT ... FROM booking_activities WHERE id = p_booking_activity_id FOR UPDATE` — this lock is what makes two simultaneous scans safe.
2. If `p_client_uuid` already exists, return that row with `already_applied: true`. Do not raise: a retry after a dropped connection is normal, not an error.
3. Sum `quantity_redeemed` for the entitlement, reversals included (they are negative, so they subtract naturally).
4. If `sum + p_quantity > quantity`: when the call is online, raise with a message naming what remains — "*Only 1 lunch remaining on WS-20260929-0042. You asked for 2.*" When the call arrives from an offline queue, insert it anyway with `is_over_redemption = true`. See section 7.4.
5. Insert, with both timestamps: `occurred_at_device` and `now()` as `synced_at`.
6. Return every figure. The device needs `remaining` to update its cache without a second round trip.

> **The unique index on `client_uuid` is not optional.** Without it, a device that loses connectivity mid-request retries and the customer is charged an entitlement twice — which, on a beach, means a customer being told their lunch is already used when it is not. This is the single defect most likely to make staff stop trusting the system.

6.3 Check-in

```
check_in_booking(p_booking_id, p_location_id, p_device_id, p_client_uuid) returns jsonb
```

- Idempotent on `client_uuid`, and idempotent in substance: checking in an already-checked-in booking returns the original time rather than overwriting it. A second scan at the jetty must not reset the arrival time.
- Refuses a cancelled booking and a booking whose `service_date` is not today, with the same readable messages the scan verdicts use.
- Sets the four columns and returns the booking's entitlement state, so the device shows the redemption list immediately after check-in without a second call.

6.4 Reversal

```
reverse_redemption(p_redemption_id uuid, p_reason text) returns jsonb
```

- Caller must be `activity_supervisor` or `admin`.
- Inserts a new row with `quantity_redeemed = -original`, `is_reversal = true`, `reverses_redemption_id`, and a mandatory reason.
- Refuses to reverse a reversal, and refuses to reverse the same redemption twice.
- Both rows stay visible on the booking forever. There is no delete button, and the UI says so rather than leaving staff hunting for one.

7 Working offline

This is the architecturally interesting part of V2 and the part most likely to be skipped under time pressure. It should not be: an island app that requires connectivity is an app that fails on exactly the days it matters.

7.1 The day manifest

At the start of a shift, on reception's wifi, the device pulls the day's manifest and stores it in IndexedDB.

```
manifest row (one per booking with service_date = today, status = 'confirmed')
  token_hash        -- sha256 of the token. NEVER the token itself.
  reference         -- WS-20260929-0042, for manual entry
  surname           -- "Smith" only; not the full client record
  adults, children, infants
  departure_time, meeting_point
  entitlements[]    -- { booking_activity_id, activity_code, activity_name,
                         quantity, already_redeemed }
  generated_at
```

Roughly 450 bytes per booking, so a hundred-booking day is about 45KB. It refreshes opportunistically whenever connectivity appears, and there is a manual refresh button.

> **Store hashes, never tokens.** The device hashes the scanned token and matches on the hash. A lost or stolen device therefore leaks a list of surnames and lunch counts, which is unpleasant but survivable — not a set of working ticket URLs, which would be a genuine breach. This costs nothing to implement and is the difference between an embarrassing incident and a serious one.

7.2 Manifest age is shown, always

A stale manifest is the failure mode staff will actually hit: a booking made at 10:00 is not in an 08:00 manifest. So the app shows its own age permanently, in words — "*Synced 08:12 · 2 hours ago*" — and turns amber past three hours. Staff can then reason about what they are seeing rather than assuming the system is wrong.

7.3 A ticket that is not in the manifest

Offline, an unknown token is ambiguous: it might be a later booking, a different day, or a forgery. The app must not guess. It shows **CANNOT VERIFY OFFLINE**, offers the manifest's age, and gives the staff member two choices:

- **Wait and retry** — the honest default when connectivity is intermittent.
- **Record a provisional redemption** — captured with the reference typed in manually, flagged `is_over_redemption = false` but `reviewed_by = null` and a provisional marker, so a supervisor sees it in the review queue. Used when a customer is standing there and the alternative is refusing a paying guest lunch.

7.4 When two devices redeem the same last lunch

Two staff, two devices, no connectivity, one remaining lunch. Both scans succeed locally. Both events are real: two people were handed food.

The system's job is not to pretend otherwise. On sync, both rows are inserted; the second is marked `is_over_redemption = true` and appears in the supervisor's review queue with both events side by side, their device labels, their staff members, and their device timestamps.

> **Why accept the second row at all.** The alternative — rejecting it on sync — loses the record of something that physically happened, and the kitchen's count will disagree with the system at the end of the day with nothing to explain the gap. Recording truth and flagging the discrepancy gives a supervisor something to act on. Silently dropping it gives them a mystery.

A supervisor resolves a flagged row by accepting it (the party genuinely received more than their entitlement — chase it or write it off) or reversing it (a mis-scan). Either way the resolution is recorded with a reason.

7.5 The sync contract

- The queue posts in order, but each item is independent and idempotent on `client_uuid`. Order is a convenience, not a requirement.
- The server returns a per-item result: `applied`, `already_applied`, `over_redemption`, or `rejected` with a reason.
- The device clears only the items the server confirmed, and keeps the rest with their failure reason visible.
- A queued item older than 48 hours is escalated in the UI rather than silently retried forever.
- Sync runs on connectivity regain, on app focus, and on a manual button. Never only on a timer.

8 Ticket lifecycle and validity verdicts

8.1 Lifecycle

```
issued ──► delivered ──► partially redeemed ──► fully redeemed
   │
   ├──► revoked        (reissued; a new ticket exists)
   └──► cancelled      (the booking was cancelled)
```

Two rules that touch V1 code and must not be missed:

- **Cancelling a booking revokes its ticket.** V1's WP-16 cancel action gains this. Without it a cancelled booking scans as valid, which is the most obvious fraud path in the system.
- **Reissuing creates a new ticket and revokes the old.** A booking has exactly one live ticket. The old token stays in the table forever so that scanning it explains itself rather than returning "unknown".

8.2 Verdicts

Seven verdicts, each with a screen a person can read at arm's length in direct sun. The colour is the message; the text is the detail.

Verdict	Colour	What the screen says, and what staff do
VALID	green	Name, party size, entitlements with remaining quantities. Redeem.
PARTLY USED	green	Same, with what has already gone: when, who, where. Redeem the remainder.
ALREADY REDEEMED	amber	The entitlement is exhausted. Shows the earlier event's time, staff member and location — which is what settles an argument on the spot.
WRONG DATE	amber	Names the date the ticket is actually for. People turn up on the wrong day; this is a real case, not an edge case. Call reception.
CANCELLED	red	Refuse and call reception. Do not show a reason — the customer may not know.
REVOKED	red	A newer ticket exists for this booking. Ask the customer to check for a later message.
NOT FOUND	red	Offer manual reference entry before concluding anything. Most "not found" results are a bad scan in bright light.

Plus one offline-only state, **CANNOT VERIFY OFFLINE** (blue), described in 7.3.

9 Delivery: WhatsApp and email

9.1 Start WhatsApp on day one of V2

> **This is a procurement item with weeks of lead time, not a coding task.** WhatsApp Business Platform access needs a verified Meta business, a phone number that is not already on a personal WhatsApp account, a provider (Meta Cloud API directly, or a BSP such as Twilio or 360dialog), and template approval that is reviewed by Meta. Template review can be same-day or can bounce repeatedly over two weeks for reasons that are not always obvious. Begin it in parallel with WP-22, or WP-32 will sit blocked while everything else is finished. Appendix A is the checklist.

Two further realities worth knowing before committing:

- **Ticket delivery is a utility template**, not marketing. Categories are priced differently and reviewed differently. Get the category right at submission or it bounces.
- **Conversations are charged per conversation, and rates vary by country.** Check the current Mauritius rate for utility conversations with your chosen provider before committing, and work out the per-booking cost — it is small, but it is a real per-transaction cost that did not exist when tickets were printed.

9.2 Every send is logged, with its result

The failure mode to design against is not a failed send — it is a send that *appears* to have worked. A receptionist who believes the customer has their ticket will not print one.

- Every attempt writes a `message_log` row before the provider is called, then updates it with the provider's message id and status.
- Provider webhooks update `delivered_at` and `read_at`. Without the webhook you know only that the API accepted the request, which is not the same as delivery.
- The booking detail screen shows the real state in words: "*WhatsApp sent 09:12 · delivered 09:12 · read 09:31*" or "*Email failed 09:12 — mailbox does not exist*". Not a green tick.
- A failed send surfaces on the day's dashboard, because the moment to fix it is while the customer is still at reception.

9.3 Email

Simpler, and it should ship first for that reason. A transactional provider with a verified sending domain and SPF and DKIM configured; the WP-18 PDF attaches directly. Get the domain verification done early too — DNS propagation plus a provider's verification queue is a day, not an hour.

10 Roles and RLS changes

10.1 The fifth role

V1 locked four roles and said a roles table was a V4 problem at the earliest. That still holds — this is one more enum value, not a permissions system.

`activity_supervisor` exists because **the person who can correct a redemption has to be standing where the redemption happened.** A mis-scanned lunch that only an admin at reception can reverse leaves a customer blocked on a beach until someone makes a phone call. The supervisor can reverse a redemption, resolve a flagged over-redemption, and register a device. They still see no prices, no totals and no commissions.

10.2 The matrix, extended

Table	admin	accountant	receptionist	act. supervisor	activity_staff
locations	all	read	read	read	read
staff_devices	all	read	—	read, register	—
ticket_redemptions	read, via fn	read	read	read, via fn, reverse	via fn only
message_log	read	read	read	—	—
tickets	all	read	read, update	—	—
`scan_ticket()`	execute	—	execute	execute	execute
`redeem_activity()`	execute	—	execute	execute	execute
`reverse_redemption()`	execute	—	—	execute	—

Activity staff still have no table grants of any kind. Everything they can do goes through two `security definer` functions that return narrow payloads. This is the boundary V1's WP-07 suite already tests; WP-24 extends that suite rather than replacing it.

11 V2 scope: in, out, and definition of done

11.1 In

QR codes on the printed ticket and the PDF · ticket revoke and reissue · cancellation invalidating a ticket · the scan endpoint with seven verdicts · the island app with camera and manual entry · **party check-in** · quantity-based redemption with partial amounts · the already-redeemed detail view · not-included rows on every entitlement display · offline manifest, queue and sync · over-redemption flagging and the supervisor review queue · redemption reversal · device registration and management · email delivery with logged results · WhatsApp delivery with templates and webhooks · delivery status on the booking · resend · **the daily operations screen** · the redemption report and kitchen reconciliation · extension of the RLS attack suite.

11.2 Out (and which phase picks it up)

Accountant dashboard and revenue reports (V3) · operator statements (V3) · cash sessions (V3) · boat and activity *capacity limits* and scheduled departures (V4) · staff assignment (V4) · participant names (V4) · automated pre-departure reminders (V4) · offline operation for *reception* (still out; V1 section 4.1 stands unchanged) · customer self-service beyond viewing their own ticket.

Moved forward from V4: the daily operations screen. The original requirements' version of it (section 21) is built almost entirely from check-in and redemption counts, both of which exist for the first time in V2. Building it here costs one work package; leaving it in V4 means the data sits unused for two phases.

11.3 Definition of done for V2

1. An island staff member scans a four-adult catamaran-plus-lunch ticket and redeems three lunches in **under 15 seconds**, one-handed, in direct sunlight, without asking anyone what to press.
2. The fourth lunch redeems an hour later, and the screen shows when, by whom and where the first three went.
3. A second scan of an exhausted entitlement shows ALREADY REDEEMED with that same detail — enough to settle a disagreement without a phone call.
4. With the device in airplane mode for a full operating day, every redemption is recorded and syncs correctly when it returns to wifi.
5. Two devices redeeming the same last lunch while offline both record it, and the supervisor's review queue shows both events with their devices and times.
6. A cancelled booking's ticket scans as CANCELLED, not as valid.
7. A reissued ticket invalidates the previous one, and scanning the old token explains why rather than returning "not found".
8. Every WhatsApp and email send has a logged provider result, and a failed send is visible on the booking and on the day's dashboard.
9. The day's redemption log reconciles against the kitchen's lunch count for ten consecutive operating days.
10. A party checks in on arrival, and the operations screen shows the trip as 28 checked in of 32 within a minute, on a second device.
11. The entitlement display shows activities the booking does not include, marked NOT INCLUDED, so a customer asking about parasailing gets an answer rather than a silence.
12. An `activity_staff` session cannot read a price, a total, a commission, an email address or a phone number, proven by a test that tries.

12 V2 build sequence — Claude Code prompts

Fifteen work packages, WP-22 to WP-36. Same rules as V1: one per session, review the diff, run the acceptance check, commit. WP-22 to WP-26 are unglamorous and must come first — the island app in WP-27 is much harder to get right if the redemption engine underneath it is still moving.

> **Start the WhatsApp application today, before WP-22.** Appendix A is the checklist. Template approval is the only item in this phase with a lead time measured in weeks, and it blocks WP-32 entirely. Everything else here is under your control; that is not.

WP-22 Schema part 5: ticketing and redemption V2

```
Migration 0006_ticketing.sql. This is the V2 foundation. Read sections 5 and 6
of the V2 build plan for column intent before writing anything.

1. Extend the role enum: ALTER TYPE app_role ADD VALUE 'activity_supervisor'.
   Note in a comment that this must be in its own migration or its own
   transaction — Postgres will not let you use a new enum value in the same
   transaction that adds it. Split the migration if needed.

2. locations
     code text unique not null, name text not null
     location_type text not null check (in ('reception','island','jetty','vessel'))
     is_active boolean not null default true

3. staff_devices
     label text not null              -- "Island tablet 1", a name a human uses
     location_id uuid references locations
     device_secret_hash text not null -- the device proves itself with this
     registered_by uuid references profiles, registered_at timestamptz
     last_seen_at timestamptz, is_active boolean not null default true

4. ticket_redemptions
     id uuid pk
     booking_id uuid not null references bookings on delete restrict
     booking_activity_id uuid not null references booking_activities on delete restrict
     activity_id uuid not null references activities
     quantity_redeemed int not null   -- negative for a reversal
     redeemed_at timestamptz not null default now()
     occurred_at_device timestamptz   -- when the device says it happened
     synced_at timestamptz not null default now()
     redeemed_by uuid references profiles
     device_id uuid references staff_devices
     location_id uuid references locations
     client_uuid uuid not null
     is_reversal boolean not null default false
     reverses_redemption_id uuid references ticket_redemptions
     is_over_redemption boolean not null default false
     is_provisional boolean not null default false
     reviewed_by uuid references profiles, reviewed_at timestamptz, review_note text
     note text
   CRITICAL: unique index on client_uuid. This is the idempotency key and it is
   the difference between a dropped connection being harmless and a customer
   being told their lunch is already used when it is not.
   Indexes on (booking_id), (booking_activity_id), (redeemed_at),
   and a partial index on (is_over_redemption) where it is true.

5. message_log
     booking_id, ticket_id, channel text check (in ('whatsapp','email'))
     template text, to_address text
     status text check (in ('queued','sent','delivered','read','failed'))
     provider text, provider_message_id text
     error_code text, error_detail text, attempt int not null default 1
     sent_by uuid, sent_at, delivered_at, read_at timestamptz
     cost_cents bigint
   Index on (booking_id), (status), (provider_message_id).

6. Extend tickets:
     revoked_at timestamptz, revoked_by uuid, revoke_reason text
     reissued_from_ticket_id uuid references tickets
     token_hash text generated always as (encode(digest(token,'sha256'),'hex')) stored
     delivered_channels jsonb not null default '[]'
   Enable pgcrypto if it is not already. Index on token_hash.

7. Extend bookings with check-in:
     checked_in_at timestamptz, checked_in_by uuid references profiles
     checked_in_device_id uuid references staff_devices
     checked_in_location_id uuid references locations

8. Audit triggers on ticket_redemptions, staff_devices, locations and
   message_log, using the existing audit_trigger() from WP-03.

9. RLS per section 10.2 of the V2 plan. The important part: activity_staff and
   activity_supervisor get NO table grants at all. Their access is entirely
   through the functions in WP-23 and WP-24. Do not add a select policy on
```

```
   bookings for them "for convenience" — that is the boundary this whole phase
   depends on.

Seed data: add three locations (Reception, Island, Jetty) and two devices to
supabase/seed.sql, plus an activity_supervisor user.

Reset, regenerate types, commit as "feat(db): WP-22 ticketing and redemption schema".
```

Done when: a local reset builds from empty, and the seed produces a supervisor account and two registered devices.

WP-23 The redemption engine V2

```
Migration 0007_redemption_functions.sql — the correctness-critical part of V2,
equivalent to V1's pricing engine. Read section 6 of the V2 build plan first and
follow it exactly.

1. redeem_activity(p_booking_activity_id, p_quantity, p_location_id, p_device_id,
                   p_client_uuid, p_occurred_at_device, p_note, p_from_queue bool)
   returns jsonb
   In order:
   a. assert the caller's role is in (admin, receptionist, activity_supervisor,
      activity_staff) or raise
   b. if p_client_uuid already exists, return that row with already_applied:true.
      DO NOT raise — a retry after a dropped connection is normal
   c. SELECT ... FROM booking_activities WHERE id = ... FOR UPDATE.
      This lock is what makes two simultaneous scans safe. Do not skip it.
   d. verify the booking is confirmed, service_date is today, and its ticket is
      not revoked; raise a readable message naming the actual state otherwise
   e. sum quantity_redeemed for this entitlement (reversals are negative and
      subtract naturally)
   f. if sum + p_quantity > quantity:
        - when p_from_queue is false, raise: "Only 1 lunch remaining on
          WS-20260929-0042. You asked for 2."
        - when p_from_queue is true, insert anyway with is_over_redemption=true.
          An offline event physically happened; the system records truth and
          flags it rather than losing it. See section 7.4.
   g. insert with both timestamps
   h. return { redemption_id, entitlement_quantity, redeemed_total, remaining,
               is_over_redemption, already_applied }

2. check_in_booking(p_booking_id, p_location_id, p_device_id, p_client_uuid)
   returns jsonb
   Idempotent in BOTH senses: idempotent on client_uuid, and checking in an
   already-checked-in booking returns the ORIGINAL checked_in_at rather than
   overwriting it. A second scan at the jetty must not reset the arrival time.
   Refuses a cancelled booking and a booking that is not for today.
   Returns the full entitlement state so the device needs no second call.

3. reverse_redemption(p_redemption_id, p_reason) returns jsonb
   - caller must be activity_supervisor or admin
   - inserts a row with quantity_redeemed = -original, is_reversal=true,
     reverses_redemption_id set, reason mandatory and non-empty
   - refuses to reverse a reversal, and refuses to reverse the same redemption
     twice
   - never deletes anything

4. resolve_over_redemption(p_redemption_id, p_action, p_note)
   where p_action is 'accept' or 'reverse'. Supervisor or admin. Sets
   reviewed_by and reviewed_at; 'reverse' also calls reverse_redemption.

Write SQL tests proving:
- two concurrent redemptions of the last remaining unit: exactly one succeeds
- the same client_uuid twice returns the same row, does not double-count
- a reversal restores the remaining count
- a queued over-redemption is inserted and flagged rather than rejected
- an online over-redemption is rejected with a message naming what remains
- check-in twice keeps the first timestamp
- a non-supervisor calling reverse_redemption is rejected

Commit as "feat(db): WP-23 redemption engine".
```

Review this diff line by line, and check for the `FOR UPDATE`. Without the row lock the engine passes every single-threaded test and silently over-redeems the moment two staff scan at once — which is precisely what happens at the lunch queue.

WP-24 The scan payload and the RLS boundary V2

```
Build the narrow read path for island staff and prove it is narrow.

1. scan_ticket(p_token text) returns jsonb — security definer, granted to
   activity_staff, activity_supervisor, receptionist and admin.
   It resolves the token and returns ONLY:
     verdict            one of VALID, PARTLY_USED, ALREADY_REDEEMED,
                        WRONG_DATE, CANCELLED, REVOKED, NOT_FOUND
```

```
     reference, client_surname, client_first_initial
     adults, children, infants
     service_date, departure_time, meeting_point
     checked_in_at, checked_in_by_name
     entitlements[]     { booking_activity_id, activity_code, activity_name,
                          quantity, redeemed, remaining,
                          last_event: { at, by_name, location_name } }
     not_included[]     { activity_code, activity_name }
   NEVER: any price, any total, commission, operator name, phone, email,
   full client name, payment status, or notes.

   The not_included array is required by the original specification: list active
   activities this booking does NOT include so a customer asking about
   parasailing gets an answer instead of a silence.

   Implement it by SELECTing explicit columns, not by building a row and
   deleting fields. A security definer function that selects bookings.* and
   then removes keys will leak the day someone adds a column.

2. Verdict logic, in this order: token not found → NOT_FOUND; ticket revoked →
   REVOKED; booking cancelled → CANCELLED; service_date <> today in Mauritius →
   WRONG_DATE (include the real date); every entitlement exhausted →
   ALREADY_REDEEMED; some redeemed → PARTLY_USED; otherwise VALID.

3. Extend tests/rls/ — the WP-07 attack suite — with an activity_staff client
   that asserts ALL of the following are denied or return zero rows:
     bookings, booking_items, payments, clients, price_rules, tour_operators,
     ticket_redemptions, message_log, tickets, audit_logs
   and that scan_ticket's JSON output contains none of the strings:
     "cents", "price", "total", "commission", "operator", "email", "phone"
   Assert the last one by scanning the serialised payload, so a future column
   addition fails the test rather than leaking quietly.

4. Add the same assertions for activity_supervisor, which gains reversal rights
   but no additional read access.

Run test:rls and commit as "feat(api): WP-24 scan payload and RLS boundary".
```

WP-25 QR codes on the ticket V2

```
Fill the square block WP-18 reserved on the ticket. This should be a small
change — if it turns into a layout rewrite, stop and tell me, because WP-18 was
built specifically to avoid that.

- Generate the QR server-side. Encode ONLY the absolute ticket URL containing
  the opaque token: https://<host>/tickets/<token>. No name, no party size, no
  price, no booking reference inside the code.
- Error correction level M, quiet zone included, rendered at a size that scans
  reliably from a phone at 30cm in bright light — test this on paper, not on a
  monitor. Minimum 30mm square on the A5 print.
- Render it into the reserved block on /tickets/[token] and into the PDF route,
  from the same component, so the two cannot drift.
- Keep the booking reference printed in large monospace directly beneath it.
  Cameras fail in salt spray and direct sun, and manual entry is the fallback
  that keeps the operation moving.
- Add a caption line: "Show this code on arrival."
- Print test: generate three tickets, print them, scan each with two different
  phones in outdoor light. Report the results. If any fails, increase the size
  before moving on.

Commit as "feat(tickets): WP-25 QR codes".
```

WP-26 Ticket lifecycle: revoke, reissue, cancellation V2

```
Make the ticket's lifecycle correct. This package changes V1 code, so read
WP-16 before starting.

1. revoke_ticket(p_ticket_id, p_reason) — admin or receptionist. Sets
   revoked_at, revoked_by, revoke_reason. Never deletes the row: a revoked token
   must still resolve so that scanning it explains itself.

2. reissue_ticket(p_booking_id, p_reason) — creates a NEW tickets row with a new
   token, sets reissued_from_ticket_id, and revokes the old one in the same
   transaction. A booking has exactly one live ticket at any time.
   The tickets table's unique constraint on booking_id must be relaxed to a
   partial unique index on (booking_id) WHERE revoked_at IS NULL.

3. CHANGE TO V1 BEHAVIOUR: cancelling a booking must now revoke its ticket.
   Update the WP-16 cancel action to call revoke_ticket with the cancellation
   reason. Without this, a cancelled booking scans as valid — the most obvious
   fraud path in the system. Add a test that cancels a booking and asserts its
   ticket scans as CANCELLED.

4. UI on the booking detail page, in the ticket panel:
```

```
   - current ticket state in words: issued / delivered / partly used / fully
     used / revoked
   - "Reissue ticket" with a reason, for a lost phone. Explain in the dialog
     that the previous ticket stops working immediately.
   - the reissue history, so a booking reissued three times shows why each time

5. Scanning a revoked token returns REVOKED with a line the staff member can act
   on: "A newer ticket was issued for this booking. Ask the customer to check
   for a later message."

Commit as "feat(tickets): WP-26 ticket lifecycle".
```

WP-27 The island app: scanner and verdicts V2

```
Build the island app at /scan. This is used by someone standing on sand, in
direct sunlight, with wet hands, holding a device in one hand. Design for that,
not for a desk.

Read section 8.2 of the V2 build plan for the seven verdicts and section 13 for
the field conditions.

Shell:
- Mobile-first, installable as a PWA with a manifest and an app icon. It will be
  pinned to a tablet home screen and must open without a browser chrome.
- A device registration flow: on first run the device asks for its label and
  registers against staff_devices, storing its secret. A staff member signs in
  on top of that at the start of a shift, while still on reception's wifi.
- A persistent status bar showing: staff name, device label, location, online or
  offline, and the manifest age in words ("Synced 08:12 · 2 hours ago", amber
  past three hours).

Scanner:
- Camera QR scanning, continuous, with a visible target frame. Use a well
  maintained library; do not hand-roll decoding.
- A large, always-visible "ENTER REFERENCE" button as an equal partner to the
  camera, not a hidden fallback. Cameras fail in salt spray and direct sun, and
  a receptionist reading a reference over the phone is a normal Tuesday.
- Manual entry accepts the digits alone ("0042") or the full reference.
- Scan feedback: a short vibration and a visible flash on decode, because in
  bright sun the screen change alone is easy to miss.

Verdict screens — one per verdict, full screen, colour-first:
  VALID (green) · PARTLY USED (green) · ALREADY REDEEMED (amber) ·
  WRONG DATE (amber) · CANCELLED (red) · REVOKED (red) · NOT FOUND (red)
- The colour fills the screen. The verdict word is the largest thing on it,
  readable at arm's length outdoors.
- ALREADY REDEEMED must show when, by whom and where the earlier event happened.
  That detail is what settles a disagreement on the spot without a phone call.
- WRONG DATE names the date the ticket is actually for.
- CANCELLED shows no reason — the customer may not know, and the staff member
  should route it to reception rather than explain it.
- NOT FOUND offers manual entry immediately, because most not-founds are a bad
  scan in bright light.

Visual rules, treat as acceptance criteria:
- Minimum touch target 56px. Minimum body text 18px. No thin font weights.
- Maximum contrast: near-black on the colour field, never mid-grey.
- No hover states, no small close buttons, no horizontal scrolling.
- Works one-handed: every primary action reachable in the bottom half of the
  screen.
- A "scan next" action is always one tap away from every verdict screen.

Commit as "feat(scan): WP-27 island app scanner and verdicts".
```

WP-28 Check-in and redemption V2

```
Build the actions on the verdict screen. Read section 3.2 of the V2 build plan:
check-in and redemption are different events and must stay that way.

CHECK IN:
- On a VALID or PARTLY USED verdict where checked_in_at is null, the primary
  action is a large CHECK IN button showing the party size: "CHECK IN · 4 people".
- Calls check_in_booking. On success the screen shows "Checked in 09:14" and the
  entitlement list becomes the focus.
- If already checked in, show the original time and who did it. Do not offer to
  check in again, and do not treat it as an error.

ENTITLEMENTS:
- One row per entitlement: activity name, quantity, remaining, and a REDEEM
  button sized for a wet thumb.
- Quantity picker for partial redemption. Default to the full remaining amount,
  because that is the common case, but make "3 of 4" two taps: a stepper, not a
  dropdown, not a keyboard.
- After redeeming, the row updates in place showing remaining, and the event
```

```
  time. No page reload, no navigation.
- An exhausted row shows USED with when, by whom and where — same information as
  the ALREADY REDEEMED verdict screen, inline.
- NOT INCLUDED rows are listed below, greyed, from the scan payload's
  not_included array. The original specification asked for these and they stop
  the "I thought parasailing was included" conversation before it starts.

Confirmation:
- Redeeming is one tap. Do NOT add a confirm dialog — it doubles the time and
  staff will learn to tap through it, which defeats the point.
- Instead make reversal easy and visible: an UNDO affordance on the row for 30
  seconds after a redemption, which calls reverse_redemption with the reason
  "Undo within 30s". After that it needs a supervisor.

Speed requirement, tested in WP-36: scan to three lunches redeemed, under 15
seconds, one-handed.

Commit as "feat(scan): WP-28 check-in and redemption".
```

WP-29 Offline operation V2

```
Make the island app work with no connectivity. Read section 7 of the V2 build
plan in full — this package follows it exactly, including the parts that look
like they could be simplified.

1. THE MANIFEST
   - A server endpoint returning today's manifest: one row per confirmed booking
     with service_date = today, containing token_hash, reference, surname,
     participant counts, departure_time, meeting_point, checked_in_at, and
     entitlements with quantity and already_redeemed.
   - STORE HASHES, NEVER TOKENS. The device hashes the scanned token with sha256
     and matches on the hash. A lost device must not leak working ticket URLs.
     This costs nothing and is the difference between an embarrassment and a
     breach.
   - Persist in IndexedDB. Roughly 45KB for a hundred bookings.
   - Refresh opportunistically on connectivity, on app focus, and on a manual
     button. Show the age permanently.

2. THE QUEUE
   - Every redemption and check-in generates a client_uuid BEFORE the attempt
     and is written to a local queue immediately.
   - Online: post at once. Offline: hold.
   - The UI must not distinguish success online from success offline. A staff
     member should not have to think about connectivity — the queue depth is
     shown in the status bar, and that is all.

3. SYNC
   - Each item posts independently and is idempotent on client_uuid. Order is a
     convenience, not a requirement.
   - The server returns per-item: applied, already_applied, over_redemption, or
     rejected with a reason.
   - Clear only confirmed items. Keep the rest with their reason visible.
   - An item older than 48 hours escalates in the UI rather than retrying
     silently forever.
   - Sync on connectivity regain, on focus, and on a manual button. Never only
     on a timer.

4. A TICKET NOT IN THE MANIFEST
   - Show CANNOT VERIFY OFFLINE (blue), with the manifest's age.
   - Offer two choices: wait and retry, or record a PROVISIONAL redemption with
     the reference typed manually, flagged is_provisional for supervisor review.
   - Do not guess. An unknown token offline might be a later booking, another
     day, or a forgery, and the app must not pretend to know which.

5. OVER-REDEMPTION ON SYNC
   - When two devices redeemed the same last unit offline, both rows are
     inserted and the second is flagged is_over_redemption. Both events
     physically happened.
   - The device shows a non-blocking notice on sync: "1 redemption flagged for
     review". It does not alarm the staff member; it is the supervisor's job.

Test by putting the device in airplane mode for a full simulated day: 30
redemptions, 12 check-ins, then reconnect and assert every row lands exactly
once with the right device timestamps.

Commit as "feat(scan): WP-29 offline operation".
```

WP-30 Supervisor tools V2

```
Build what the activity_supervisor needs, at /scan/supervisor. This is a
separate view inside the island app, not an admin screen — the supervisor is
standing on the same beach.

1. REVIEW QUEUE — the reason this role exists.
```

```
   - Every redemption where is_over_redemption or is_provisional is true and
     reviewed_at is null.
   - Each item shows BOTH conflicting events side by side where there is a
     conflict: time, quantity, staff member, device label, location, and the
     device timestamp versus the sync timestamp. That comparison is what lets a
     supervisor work out what actually happened.
   - Two actions: ACCEPT (the party genuinely received more than their
     entitlement — chase it or write it off) or REVERSE (a mis-scan). Both
     require a note. Both call resolve_over_redemption.
   - An unreviewed item older than 24 hours is highlighted.

2. REVERSAL — reverse any redemption from the last 24 hours with a mandatory
   reason. Beyond 24 hours it needs an admin. Both rows stay visible forever and
   the UI says so, so nobody goes looking for a delete button.

3. DEVICE MANAGEMENT
   - List devices: label, location, last seen, queue depth at last sync, active.
   - Register a new device, and DEACTIVATE a lost one. Deactivating must
     immediately stop that device's functions from succeeding — test this, do
     not assume the session expires.

4. SHIFT HANDOVER — a simple screen showing this device's activity for the
   current shift: check-ins, redemptions by activity, anything flagged. It is
   what one staff member shows the next one at changeover.

Commit as "feat(scan): WP-30 supervisor tools".
```

WP-31 Email delivery V2

```
Ship email first, because it is the simpler of the two channels and it proves
the message_log plumbing that WhatsApp will reuse.

1. Provider: Resend or similar. Verify the sending domain with SPF and DKIM.
   Start the DNS work now — propagation plus a provider verification queue is a
   day, not an hour.

2. sendTicketEmail(bookingId, { resend: boolean }) as a Server Action:
   - writes a message_log row with status 'queued' BEFORE calling the provider
   - calls the provider, updates the row with provider_message_id and 'sent',
     or 'failed' with the error code and detail
   - attaches the PDF from the WP-18 /tickets/[token]/pdf route — one source of
     truth for the layout, not a second template
   - never throws to the caller: a failed email must leave a logged row and a
     visible failure, not a stack trace

3. The email body, following the original specification:
     YOUR BOOKING IS CONFIRMED
     John Smith
     29 September 2026 · 08:30
     CATAMARAN + LUNCH
     Meeting point: ...
     Booking: WS-20260929-0042
     [QR code inline]
     Please show this ticket when requested.
   Plain text and HTML parts, both. The QR is inline in the HTML and also in the
   attached PDF, because many clients block remote images.

4. Provider webhooks at a route handler: update delivered_at, and bounced or
   complained to 'failed' with the reason. Verify the webhook signature.

5. Never send to an empty address. A booking without an email shows the email
   button disabled with "No email address on this booking" rather than failing
   after the click.

Commit as "feat(delivery): WP-31 email delivery".
```

WP-32 WhatsApp delivery V2

```
Add WhatsApp. This package is blocked until template approval comes back — check
Appendix A of the V2 build plan before starting, and if the template is still
pending, skip to WP-34 and return to this.

1. Provider: WhatsApp Business Platform, through Meta Cloud API directly or a
   BSP. Credentials live in Vercel server environment only and are used from a
   Supabase Edge Function or a server-only module — never in a browser bundle.

2. The approved utility template, matching the original specification:
     Your booking is confirmed.
     📅 Date: {{1}}
     ⏱ Departure: {{2}}
     📍 Meeting point: {{3}}
     🚤 Activity: {{4}}
     🍽 Lunch: {{5}}
     Please show your digital ticket upon arrival.
```

```
     Booking: {{6}}
   Followed by the ticket PDF or an image of the QR as a media message.
   Submit as UTILITY, not MARKETING — the categories are priced and reviewed
   differently and a wrong category is the most common rejection reason.

3. sendTicketWhatsApp(bookingId, { resend }) mirroring WP-31 exactly: queued row
   first, provider call, update with provider_message_id, never throws.

4. Webhooks: update sent / delivered / read, and failed with the error code.
   Meta's error codes are specific — store the code, not a paraphrase, because
   "invalid number" and "user has not opted in" need different responses from
   reception. Verify the webhook signature.

5. Opt-in and number validity:
   - only send to a phone_e164 that passed validation at booking time
   - a failed send with an invalid-number error sets a flag on the client so
     reception sees it next time rather than re-failing
   - record cost_cents per conversation if the provider reports it

6. Never send to an empty or invalid number. Disabled button with the reason,
   as with email.

Commit as "feat(delivery): WP-32 WhatsApp delivery".
```

WP-33 Delivery status and the ticket panel V2

```
Make delivery visible, so reception knows whether the customer actually has
their ticket. This is the package that stops the whole feature being a lie.

1. On the booking detail page, a ticket panel showing:
   - ticket state, and the QR itself at a size reception can hold up to a phone
   - delivery history from message_log, in words and in full:
       "WhatsApp sent 09:12 · delivered 09:12 · read 09:31"
       "Email failed 09:12 — mailbox does not exist"
     Not a green tick. A tick cannot express "sent but never delivered", which
     is the state that matters.
   - buttons: Print, Send WhatsApp, Send email, Reissue. Each disabled with a
     reason when it cannot apply.
   - a resend confirms if a successful send already exists ("This was delivered
     at 09:12. Send again?").

2. On the WP-16 confirmation screen, enable the two buttons that V1 shipped
   disabled, and show their result inline without leaving the screen.

3. On /today, a FAILED DELIVERIES strip listing today's bookings whose latest
   send failed, with one-tap resend. The moment to fix a failed ticket is while
   the customer is still at reception, which is why this belongs on the day
   screen and not in a report.

4. Auto-send on booking creation, controlled by a new app_settings key
   auto_send_ticket ('off' | 'email' | 'whatsapp' | 'both'). Default 'off' —
   turn it on once the channels are proven, not on day one.

Commit as "feat(delivery): WP-33 delivery status and ticket panel".
```

WP-34 The daily operations screen V2

```
Build /operations — the real-time view the original specification asked for in
its section 21. It is built almost entirely from check-in and redemption data,
which is why it belongs in V2 rather than V4.

Structure, following the specification:

  TODAY · 29 SEPTEMBER

  08:00  CATAMARAN        32 PEOPLE
  09:00  PARASAILING      12 PEOPLE
  09:30  UNDERSEA WALK     8 PEOPLE
  10:00  PRIVATE BOAT      6 PEOPLE

- Grouped by departure_time then activity, from booking_activities joined to
  today's confirmed bookings.
- Each row: time, activity, total people entitled, and a check-in progress
  indicator.

Clicking a row opens the detail:

  CATAMARAN — 08:00
  32 PEOPLE
  ✓ 28 checked in      ○ 4 pending
  Lunch: 32   Redeemed: 26
  Parasailing: 8

- Checked in versus pending comes from bookings.checked_in_at.
```

```
- Per-activity entitled versus redeemed comes from booking_activities and
  ticket_redemptions.
- A list of the parties in that group: surname, party size, checked in or not,
  and what they still have outstanding. Receptionist and above see the booking
  reference and can click through; activity_staff do not get this screen at all.

Rules:
- Refresh on window focus and poll every 30 seconds. Two people watching this on
  two devices must not disagree for long.
- The pending count is the operationally useful number — it is who has not
  arrived yet, ten minutes before a boat leaves. Make it the most prominent
  thing on the row.
- A party checked in but with nothing redeemed by 13:00 is highlighted: it is
  usually someone who missed lunch, which is recoverable if anyone notices.
- Roles: admin, receptionist, accountant and activity_supervisor.

Commit as "feat(ops): WP-34 daily operations screen".
```

WP-35 Redemption reporting and reconciliation V2

```
Build the reports that let V2 be trusted, and that the V2 to V3 gate is measured
against.

1. /reports/redemptions — for a date range:
   - per activity: entitled, redeemed, outstanding, redemption rate
   - per location and per device
   - per staff member, which is a management tool and should be presented as one
     without editorialising
   - flagged rows: over-redemptions and provisionals, with their resolution
   - Excel export, consistent with WP-17's export

2. THE KITCHEN RECONCILIATION — the specific report the V2 to V3 gate needs:
   for a single day, lunches entitled versus lunches redeemed versus a figure
   the kitchen enters manually, with the variance and an explanation field.
   One row per operating day, so ten days of green is visible at a glance.
   Add a small form for the kitchen's count; do not require a spreadsheet.

3. Add to /today's KPI row: checked in, redeemed today, flagged for review.
   Keep it to three numbers — the day screen is reception's, not management's.

4. A no-show view: today's confirmed bookings with no check-in by a configurable
   cut-off. This is the first time the system can tell a no-show from a
   cancellation, which changes what the day's figures mean. Do not auto-set
   status to 'no_show' — surface it and let a human decide, because a party that
   arrived late and was never scanned looks identical to one that never came.

Commit as "feat(reports): WP-35 redemption reporting".
```

WP-36 Field trial and hardening V2

```
Harden V2 before it goes to the island.

Automated tests:
- Playwright: scan a seeded ticket, check in, redeem 3 of 4 lunches, assert
  remaining is 1; scan again and assert PARTLY USED with the earlier event's
  detail; redeem the last and assert ALREADY REDEEMED.
- Cancelled booking scans as CANCELLED. Reissued ticket: old token scans as
  REVOKED, new one as VALID.
- Wrong-date ticket scans as WRONG_DATE naming the real date.
- Offline simulation: airplane mode, 30 redemptions and 12 check-ins, reconnect,
  assert every row lands exactly once.
- Concurrency: two clients redeem the last unit simultaneously online; assert
  exactly one succeeds and the other gets a readable message.
- Idempotency: post the same client_uuid five times; assert one row.
- Run the extended RLS suite from WP-24 and confirm it is green.

Field conditions — actually test these, they are not hypothetical:
- Outdoor legibility: take the device outside at midday and photograph each
  verdict screen. If a verdict is not readable at arm's length, increase the
  size or the contrast.
- One-handed operation: complete a full scan-to-redeem holding the device in one
  hand only.
- Wet-hand operation: test with damp fingers. Capacitive touch degrades; targets
  may need to grow.
- Battery: run the app with the camera active for four hours and report the
  drain. If a shift cannot be completed on one charge, say so now — it is a
  hardware decision, not a code one.

The trial, with real staff:
- Two weeks of live redemptions alongside whatever is used today.
- Daily kitchen reconciliation from WP-35.
- A written note of every time a staff member had to ask someone what to do.
  That list is the real defect log.
```

```
Then write docs/USER_GUIDE_ISLAND.md — one page, screenshots, what each verdict
means, what to do when the camera will not read, what to do when the device says
offline, and who to call.

Commit as "test: WP-36 field trial and hardening".
```

— **Gate V2 → V3.** Island staff have redeemed on live bookings for two weeks and the redemption log reconciles against the kitchen's lunch count. —

13 Hardware and field conditions

V2 is the first phase where the software meets weather. These decisions cost money and have lead times, so make them while WP-22 is being built.

Item	Recommendation and reasoning
Device	A mid-range Android tablet or large phone, 8" or above, bought as a **dedicated device** rather than using staff phones. Dedicated devices make `staff_devices` accountability real, survive being handed between shifts, and mean a lost device is not also someone's personal phone. Budget for two plus a spare.
Screen	**Brightness matters more than resolution.** Aim for 500 nits or better. A cheap tablet at 300 nits is unreadable on a beach at noon, and no amount of contrast work in WP-27 will fix it.
Camera	Autofocus, not fixed focus. Fixed-focus cameras fail on QR codes at close range, which is exactly how they will be used.
Case	IP67 rated, with a hand strap or lanyard. Salt water is the failure mode, not drops — and salt corrodes ports over weeks rather than failing at once. Prefer a case that covers the charging port.
Power	Confirm a full operating day on one charge with the camera active (WP-36 measures it). If not, a power bank per device, not a charging cable on the island.
Connectivity	Assume none on the island. If there is a 4G signal, treat it as a bonus that makes sync more frequent, never as a dependency. The offline path in WP-29 is the primary path.
Spare	One charged spare device at reception, registered and ready. A failed device mid-morning otherwise stops redemption entirely.

14 Operations runbook additions

Task	How
Customer lost their phone	Reissue the ticket from the booking detail page with a reason. The old token stops working immediately and scans as REVOKED.
Ticket redeemed by mistake	Undo within 30 seconds from the row itself. Beyond that, a supervisor reverses it with a reason. Both rows stay visible.
Camera will not read the code	Type the reference. It is printed in large monospace under the QR for exactly this reason.
Device says offline all day	Keep working — redemptions queue. Check the queue depth in the status bar. Sync at reception at the end of the shift and review anything flagged.
A ticket is not in the offline manifest	Wait and retry if connectivity is intermittent. If the customer is standing there, record a provisional redemption; a supervisor reviews it.
Two staff redeemed the same lunch	It will appear in the supervisor review queue after sync with both events side by side. Accept or reverse, with a note.
Device lost or stolen	Deactivate it in supervisor tools immediately. It holds no tokens and no prices, only surnames and lunch counts for one day.
Customer says they never got their ticket	Check the delivery history on the booking. It shows sent, delivered and read separately. Resend or print.
WhatsApp sends start failing	Check the error code in `message_log`, not the status. Template rejection, number quality rating and billing failures all look like "failed" and need different fixes.

15 Open questions and risks

Question	Why it matters, and when to answer it
Is there any signal on the island?	Measure it before WP-29, standing where staff will stand, at the time of day they will be there. It does not change whether offline is built — it is — but it changes how often sync happens, and therefore how long a flagged conflict stays invisible.
Who is the supervisor, in practice?	The role only works if someone with it is physically present during operating hours. If the answer is "the owner, sometimes", reversal beyond 30 seconds will be a bottleneck. Decide before WP-30 and consider giving two senior staff the role.
What is the WhatsApp cost per booking?	Utility conversations are charged per conversation at a country rate. Work out the per-booking cost with your provider before turning on auto-send. It is small, but it is a per-transaction cost that printing did not have.
Do customers actually open WhatsApp tickets?	Measure it: `read_at` in `message_log` tells you. If read rates are poor, printing is not obsolete and the desk should keep doing both.
Should check-in happen at reception or at the boat?	V2 allows either — it records the location. Watch where it actually happens for two weeks before writing it into a procedure, because the answer determines what "pending" means on the operations screen.
What happens to a no-show's entitlements?	Currently nothing: they stay unredeemed. That is correct for V2. Whether a no-show is refundable, transferable or forfeit is a commercial decision that V3's reporting will make visible and that someone needs to settle.
Thermal printers?	The original notes raised them. Leave it: A5 printing works, and a thermal receipt cannot carry a QR at a reliable scanning size without becoming as large as the A5 anyway.

16 Appendix A — WhatsApp procurement checklist

Start this on the same day as WP-22. Every item below has a queue behind it that is not yours to control.

#	Step	Lead time
1	Create or confirm a Meta Business Manager account for the company.	Hours
2	Complete Meta business verification — company registration documents, address proof. This is the step that most often stalls.	Days to weeks
3	Choose a provider: Meta Cloud API directly, or a BSP (Twilio, 360dialog and others). A BSP costs more per message but removes most of the integration work.	Hours
4	Obtain a phone number that is **not** currently registered to a personal or business WhatsApp account. Migrating an existing number is possible but adds steps and downtime.	Days
5	Register the number and complete the display-name review.	Days
6	Submit the ticket template as **UTILITY**, with all six variables and the media header. Wrong category is the most common rejection.	Hours to days, repeatable
7	Confirm the per-conversation rate for Mauritius with the provider, and calculate the cost per booking.	Hours
8	Set up and verify the webhook endpoint for delivery status, with signature verification.	Hours
9	Understand the quality rating: too many blocks or unread messages degrades it and eventually limits sending. Only message customers who gave you their number for this purpose.	Ongoing

Email's equivalent is one item — verify the sending domain with SPF and DKIM — and takes a day. Ship email first (WP-31) so the delivery plumbing is proven before WhatsApp's approvals land.

17 Appendix B — Enumerations added in V2

Enum or constrained column	Values
`app_role` *(extended)*	admin · accountant · receptionist · activity_staff · **activity_supervisor**
`location_type`	reception · island · jetty · vessel
scan `verdict`	VALID · PARTLY_USED · ALREADY_REDEEMED · WRONG_DATE · CANCELLED · REVOKED · NOT_FOUND · *(client-only)* CANNOT_VERIFY_OFFLINE
message `channel`	whatsapp · email
message `status`	queued · sent · delivered · read · failed
sync result	applied · already_applied · over_redemption · rejected
review `action`	accept · reverse
`auto_send_ticket` setting	off · email · whatsapp · both

18 Appendix C — The island field card

Print this at A6, laminate it, and attach it to each device. It is not documentation; it is what someone reads with a customer waiting.

```
──────────────────────────────────────────────
SCANNING — QUICK CARD
──────────────────────────────────────────────

GREEN   VALID / PARTLY USED
        → Check in, then redeem what they take.

AMBER   ALREADY REDEEMED
        → Screen shows when, who and where.
          Show the customer. No call needed.

AMBER   WRONG DATE
        → Screen shows their real date.
          Send them to reception.

RED     CANCELLED or REVOKED
        → Do not serve. Call reception.
          Do not explain the reason yourself.

RED     NOT FOUND
        → Try typing the reference first.
          Most of these are a bad scan in sun.

BLUE    CANNOT VERIFY OFFLINE
        → Wait and retry if you can.
          If the customer is waiting, use
          PROVISIONAL and tell the supervisor.

──────────────────────────────────────────────
CAMERA WON'T READ  → ENTER REFERENCE. Digits
only are fine: 0042
SAYS OFFLINE       → Keep working. It queues.
WRONG TAP          → UNDO on the row, 30 secs.
                     After that: supervisor.
──────────────────────────────────────────────
Reception: _____________________
Supervisor: ____________________
──────────────────────────────────────────────
```

19 Appendix D — Acceptance test checklist for V2

Run this on the island, with real staff, on real devices, before the trial is called finished. Every line is pass or fail.

#	Check	Pass
1	A printed QR scans first time from 30cm, outdoors at midday, on two different devices.	
2	Scan to check-in to three lunches redeemed, one-handed, in under 15 seconds.	
3	The fourth lunch redeems later and shows when, who and where the first three went.	
4	A fully redeemed entitlement shows ALREADY REDEEMED with that detail, legible at arm's length in sun.	
5	NOT INCLUDED activities are listed, so "is parasailing included?" is answered on screen.	
6	Checking in twice keeps the original arrival time and does not read as an error.	
7	A cancelled booking scans as CANCELLED.	
8	A reissued ticket: the old token scans as REVOKED, the new one as VALID.	
9	A ticket for tomorrow scans as WRONG DATE and names the date.	
10	Two staff redeeming the last lunch at the same time, online: one succeeds, the other gets a readable message.	
11	A full day in airplane mode: every redemption and check-in syncs exactly once.	
12	Two devices redeeming the same last unit offline: both recorded, the second flagged, both visible in the review queue with their devices and times.	
13	A supervisor resolves a flagged redemption and the resolution is recorded with a reason.	
14	An UNDO within 30 seconds reverses cleanly; both rows remain visible.	
15	A deactivated device immediately stops being able to redeem.	
16	The device holds no usable ticket tokens — verified by inspecting IndexedDB.	
17	An `activity_staff` session cannot read a price, total, commission, email or phone. Proven by the extended RLS suite.	
18	A WhatsApp ticket arrives, and the booking shows sent, delivered and read as separate states.	
19	An email to a bad address shows as failed on the booking and on the day's failed-deliveries strip.	
20	The operations screen shows checked in versus pending for a trip, and updates within a minute on a second device.	
21	The day's redeemed lunch count matches the kitchen's own count, ten operating days running.	
22	A staff member who has never used it completes a scan and redemption after five minutes of instruction, using the field card only.	
23	A device completes a full operating shift on one charge with the camera in use.	
24	Every redemption row has a named actor, a device and a location.	

End of document · Watersports Operations Platform · V2 build plan v1.0 · 29 September 2026